Predictive Modeling and Analysis of Traffic Accident Severities Using Machine Learning Algorithms

Abstract—Traffic accidents represent a major global public safety issue, leading to substantial property loss, physical injuries, and fatalities. Predicting accident severities and identifying key contributing environmental factors are essential steps toward designing proactive traffic safety measures. This paper introduces an analytical framework that utilizes Chicago traffic crash data to model and predict accident severity patterns. Data cleaning, target binarization, and feature engineering transformations are carried out on categorical environmental descriptors. Four machine learning algorithms are trained and evaluated: Random Forest, Logistic Regression, Decision Tree, and K-Nearest Neighbors (KNN). Empirical evaluations show that the Random Forest ensemble model outperforms the other models, achieving a peak validation accuracy of 84.19%. The findings emphasize that predictive modeling can serve as a dependable instrument for municipal decision-making and infrastructure refinement.

***Keywords*—Traffic Accident Analysis, Classification, Random Forest, Machine Learning, Predictive Maintenance.**

I. Introduction

Urban expansion and increasing vehicular density have led to a steady rise in the complexity of road traffic safety management. Traffic accidents cause significant socioeconomic harm, strain healthcare systems, and lead to substantial infrastructure repair expenses. Conventional descriptive statistics can summarize historical incidents, but they fall short in uncovering hidden non-linear correlations or providing real-time predictive support for emergency dispatchers and municipal planning boards.

Predictive intelligence can help municipal authorities implement preemptive measures. Machine learning classification models can analyze historical data to determine whether an active traffic collision is likely to involve physical injuries or remain limited to vehicle property damage.

This study presents an end-to-end data science methodology applied to an extensive urban crash database. The primary objectives include:

1. Developing a clean, structural preprocessing pipeline capable of handling high-cardinality categorical variables, missing records, and variable scaling.
2. Formulating an optimized binary target variable framework focused on isolating severe physical injuries from non-injury events.
3. Evaluating the performance of multiple classifier architectures under standardized train-test splits to isolate the most robust predictive system.

II. Methodology

The structural design of the predictive pipeline follows standard machine learning engineering steps: data acquisition, exploratory manipulation, preprocessing, feature transformation, model compilation, and evaluation metrics.

A. Dataset Selection and Structural Characteristics

The empirical base of this study relies on a comprehensive urban traffic accident dataset containing multi-column fields for weather variations, light configurations, active roadway conditions, and injury breakdowns. Primary attributes extracted for classification include:

- traffic_control_device, weather_condition, lighting_condition
- first_crash_type, trafficway_type, alignment
- roadway_surface_cond, road_defect

- Temporal vectors: crash_hour, crash_day_of_week, crash_month

B. Preprocessing and Feature Engineering

To ensure optimal convergence of gradient-based and distance-dependent algorithms, raw fields underwent several transformations:

1. **Handling Irrelevant Attributes:** Non-predictive identification markers (e.g., precise geographical coordinate tags, serial reference indices) were excluded from training.
2. **Target Variable Construction:** The column crash_type originally contained descriptions indicating either visible injury presence or simple property damage (NO INJURY / DRIVE AWAY). This column was binarized into a single structured label target variable, where 1 represents an injury-producing incident and 0 signifies property damage only.
3. **Categorical Vectorization:** High-cardinality nominal values within environmental variables (e.g., weather_condition, lighting_condition) were mapped into numerical scales using scikit-learn's LabelEncoder module.

C. Predictive Model Architectures

Four classifiers were chosen to evaluate performance across ensemble methods, parametric functions, non-parametric boundaries, and instance-based proximity computations:

1. **Random Forest Classifier:** An ensemble bootstrap aggregation method that assembles multiple distinct decision trees to reduce variance and mitigate over-fitting.
2. **Logistic Regression:** A baseline parametric model that uses a logistic sigmoid function to output probabilities for binary classification.
3. **Decision Tree Classifier:** A structural non-parametric model that recursively partitions data samples based on feature split points optimized via information gain or Gini impurity criteria.
4. **K-Nearest Neighbors (KNN):** An instance-based learning model that classifies a target data point by calculating a plurality vote among its closest spatial neighbors within the feature space.

III. Experimental Results and Analysis

A. Experimental Setup

The cleaned matrix was split using a standardized 80:20 partition, allocating 80% of historical instances to training sets and retaining 20% for validation tests. Random seeds were fixed to ensure reproducibility across executions. Models were evaluated using the global classification accuracy metric:

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Samples}}$$

B. Comparative Performance Analysis

Following model training and hyperparameter optimization, test sets were processed by each algorithm. The observed validation accuracy scores are summarized in Table I.

TABLE I: Predictive Performance Comparison

Rank	Classifier Architecture	Validation Accuracy Score
1	**Random Forest Ensemble**	**84.19%**
2	Logistic Regression	83.16%
3	Decision Tree Classifier	79.23%

Rank	Classifier Architecture	Validation Accuracy Score
4	K-Nearest Neighbors (KNN)	74.52%

The Random Forest ensemble achieved the highest performance with an accuracy of 84.19%. This indicates that tree-based bootstrap aggregation effectively captures complex interactions among mixed temporal and environmental variables without overfitting.

Logistic Regression also performed well, reaching 83.16% accuracy, which suggests a strong linear separability within the engineered categorical features. The standard Decision Tree model recorded lower accuracy (79.23%), likely due to structural over-fitting on high-depth nodes. K-Nearest Neighbors registered the lowest baseline accuracy (74.52%), primarily because distance computations degrade when applied to encoded high-cardinality categorical feature spaces.

IV. Conclusion and Future Work

This study developed an IEEE-compliant workflow to predict traffic accident severity by leveraging structural categorical preprocessing and multiple machine learning architectures. The empirical analysis demonstrates that ensemble learning techniques, particularly Random Forest, provide robust predictive performance, achieving an accuracy of 84.19%. These predictive models offer valuable diagnostic insights for urban development boards, traffic management offices, and emergency response teams.

Future work will focus on integrating real-time telemetry inputs, applying advanced synthetic oversampling methods (such as SMOTE) to address minority class injury distribution imbalances, and deploying deeper neural network architectures to further enhance predictive performance.

References

[1] Chicago Data Portal, "Traffic Crashes - Crashes," City of Chicago, Department of Transportation. [Online Available Data Source].

[2] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[4] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61.